ESP32 LED Blinking Using ESP-IDF and Embedded C

Complete project documentation: installation, Linux commands, source code, build process and Wokwi simulation

1. Project Overview

Project title: LED Blinking Using ESP32 and ESP-IDF

Microcontroller: ESP32-WROOM-32 (simulated using an ESP32 DevKit model in Wokwi)

Development environment: Xubuntu Linux; programming language: Embedded C; framework: ESP-IDF; build system: CMake and Ninja; simulator: Wokwi; editor: Visual Studio Code.

Objective: Configure ESP32 GPIO 2 as an output and repeatedly switch it HIGH and LOW so that an LED turns ON for approximately one second and OFF for approximately one second.

2. Software and Tools

Tool	Purpose
Xubuntu	Linux development environment
Git	Download ESP-IDF source code
Python 3	Run ESP-IDF installation scripts and tools
CMake	Configure the project build
Ninja	Run the build process
ESP-IDF	Espressif's development framework for ESP32
VS Code	Edit project files and run the Wokwi extension
Wokwi	Simulate the ESP32 circuit

ESP-IDF provides drivers and software components for peripherals such as GPIO, UART, SPI, I2C, ADC, PWM, Wi-Fi and Bluetooth. ESP-IDF applications also use FreeRTOS.

3. Installing ESP-IDF in Xubuntu

Update package information:

```
sudo apt update
```

Install the required dependencies:

```
sudo apt install -y git wget flex bison gperf python3 python3-pip python3-venv cmake ninja-build ccache libffi-dev libssl-de
```

Create and enter the ESP-IDF workspace:

```
mkdir -p ~/esp
cd ~/esp
```

Download ESP-IDF and its submodules:

```
git clone --recursive https://github.com/espressif/esp-idf.git
```

Enter the ESP-IDF directory and install the ESP32 tools:

```
cd ~/esp/esp-idf
./install.sh esp32
```

Activate the ESP-IDF environment in the current terminal:

```
. ~/esp/esp-idf/export.sh
```

Check the installed version:

```
idf.py --version
```

The export command sets environment variables and adds ESP-IDF tools to the current terminal's PATH. Run it again in each new terminal session before using idf.py.

4. Project Structure

The project was created from an ESP-IDF example project and its application source file was named LED_BLINKING.c.

```
ESP32_First_Project_1/
■■■ build/
■■■ main/
■   ■■■ CMakeLists.txt
■   ■■■ LED_BLINKING.c
■■■ CMakeLists.txt
■■■ sdkconfig
■■■ sdkconfig.old
■■■ diagram.json
■■■ wokwi.toml
```

main/LED_BLINKING.c contains the application code. main/CMakeLists.txt tells ESP-IDF which source file to compile. The top-level CMakeLists.txt defines the project build. sdkconfig stores project configuration. build/ contains generated build outputs. diagram.json defines the Wokwi circuit, and wokwi.toml points Wokwi to the firmware files.

5. Embedded C Program

```
#include <stdio.h>
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
    gpio_reset_pin(LED_PIN);
    gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

    while (1)
    {
        gpio_set_level(LED_PIN, 1);
        printf("LED ON\\n");
        vTaskDelay(pdMS_TO_TICKS(1000));

        gpio_set_level(LED_PIN, 0);
        printf("LED OFF\\n");
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

Line-by-line explanation:

Code / concept	Purpose
#include <stdio.h>	Provides printf() for serial log messages.
#include "driver/gpio.h"	Provides ESP-IDF GPIO functions.
#include "freertos/FreeRTOS.h" and task.h	Provide FreeRTOS definitions and task-delay functions.
#define LED_PIN GPIO_NUM_2	Names GPIO 2 as LED_PIN so the pin is easy to refer to.
void app_main(void)	ESP-IDF application entry point.
gpio_reset_pin(LED_PIN)	Resets the selected GPIO configuration.
gpio_set_direction(..., GPIO_MODE_OUTPUT)	Configures GPIO 2 as an output.
while (1)	Repeats the LED sequence continuously.
gpio_set_level(LED_PIN, 1)	Drives GPIO 2 HIGH; the LED turns ON in the stated circuit.
printf(...)	Prints a message; it does not control the LED.
vTaskDelay(pdMS_TO_TICKS(1000))	Suspends the current FreeRTOS task for approximately 1000 ms.
gpio_set_level(LED_PIN, 0)	Drives GPIO 2 LOW; the LED turns OFF.

The FreeRTOS delay yields the CPU to other tasks instead of using a busy-wait loop. The delay is approximate and depends on the FreeRTOS tick configuration.

6. CMake Configuration

Because the source file is named LED_BLINKING.c, main/CMakeLists.txt must register that exact filename:

```
idf_component_register(
    SRCS "LED_BLINKING.c"
    INCLUDE_DIRS "."
)
```

The filename is case-sensitive on Linux. The source filename and the name registered in CMake must match exactly. Also ensure there is only one intended app_main() implementation in the component.

7. Building the Firmware in Linux

Open a Xubuntu terminal and change to the project directory. Activate ESP-IDF, set the chip target, and build:

```
. ~/esp/esp-idf/export.sh
idf.py set-target esp32
idf.py build
```

CMake configures the project, Ninja runs the build, the compiler compiles the C source and the linker creates the application image. A successful build displays 'Project build complete.'

Check the generated files (the exact application filename should match the actual project name):

```
ls build/*.elf
ls build/*.bin
ls build/flasher_args.json
```

The .elf file contains executable and debugging information. The .bin file is the application binary image. flasher_args.json contains flash layout and image information used by flashing workflows. Do not assume the .bin filename from the .elf filename; check the build directory.

8. Wokwi Circuit

The simulated circuit used an ESP32 DevKit, a breadboard, a red LED, a 330-ohm resistor and jumper wires.

Connection	Description
ESP32 GPIO 2	Connects to one end of the 330-ohm resistor
Other resistor end	Connects to LED anode (A)
LED cathode (C)	Connects to ESP32 GND

The resistor limits current through the LED. With this active-high wiring, GPIO 2 HIGH turns the LED ON and GPIO 2 LOW turns it OFF.

9. Wokwi Configuration and Simulation

Place diagram.json in the project root, alongside the top-level CMakeLists.txt. Create wokwi.toml in the same folder. Use the actual .bin and .elf names from the build directory. Example only:

```
[wokwi]
version = 1
firmware = "build/ESP32_First_Project_1.bin"
elf = "build/ESP32_First_Project_1.elf"
```

If the generated binary has a different name, update the firmware path to that exact filename. Install the Wokwi Simulator extension in VS Code and follow its license activation prompt. Open the project root folder in VS Code, then use the Command Palette (Ctrl+Shift+P) and select Wokwi: Start Simulator. If the command is missing, confirm the extension is installed and enabled, then reload VS Code.

10. Result and Verification

The Wokwi simulation boot log showed that the ESP32 bootloader loaded the application and called app_main(). The LED blinked after the correct source file was registered in CMake and the FreeRTOS header capitalization was corrected.

```
LED ON
LED OFF
```

```
LED ON
LED OFF
```

Expected behaviour: LED ON for approximately one second, then LED OFF for approximately one second, repeating continuously.

11. Linux Commands Reference

Command	Purpose
sudo apt update	Refresh package information
sudo apt install ...	Install Linux packages
mkdir -p ~/esp	Create the ESP-IDF workspace
cd path	Change directory
pwd	Print current directory
ls / ls -l	List files; -l shows details
cat file	Print a file's contents
nano file	Edit a file in the terminal
cp source destination	Copy files
mv old new	Move or rename files
find path -name name	Search for a file
git clone --recursive URL	Clone repository and submodules
. ~/esp/esp-idf/export.sh	Activate ESP-IDF in current shell
idf.py --version	Show ESP-IDF version
idf.py set-target esp32	Set target chip
idf.py build	Build the project
idf.py fullclean	Remove generated build output before a fresh build
idf.py flash	Flash firmware to a connected physical board; not needed for Wokwi simulation

12. Embedded Concepts Practised

GPIO configuration; digital HIGH and LOW states; output control; Embedded C; ESP-IDF application structure; FreeRTOS task delay; infinite loops; CMake source registration; firmware compilation; and virtual hardware simulation.

13. Conclusion

This project introduced the basic embedded firmware workflow using ESP-IDF and Embedded C. It demonstrated how to configure an ESP32 GPIO pin, control an LED, use a FreeRTOS task delay, compile firmware in Xubuntu and test the result in Wokwi without a physical development board. The project provides a foundation for future work with push buttons, PWM, ADC, UART, SPI, I2C, interrupts and FreeRTOS multitasking.

14. Important Project-Specific Notes

Source file used: main/LED_BLINKING.c. GPIO used: GPIO 2. ESP-IDF workspace: ~/esp/esp-idf. Windows shared folder: C:\\ESP32_Shared. Project folder: C:\\ESP32_Shared\\ESP32_First_Project\\ESP32_First_Project_1.

A successful build alone does not prove that the intended source file was compiled. Confirm that main/CMakeLists.txt registers LED_BLINKING.c and rebuild after changing source or CMake configuration. If the simulator shows 'app_main returned', inspect the compiled source and ensure the intended app_main() contains the infinite loop.